JandHInventory - Security & Bug Audit

Project: Taylored Expressions Inventory Manager (.NET 8 WPF, EF Core, SQL Server)
Date: 2026-04-26
Scope: Source tree at `D:\JandHInventory` (excluding bin/, obj/, generated XAML)
Method: Static review of Services/, ViewModels/, Views/, Data/, Converters/, Models/. Focus areas: SQL injection, path traversal, process execution, async/UI-thread safety, resource lifecycle, hardcoded secrets, logic gaps in the wizard.

Executive Summary

The application is generally well-structured. EF Core is used everywhere database access happens, so SQL injection risk is minimal. URL launches in the Wishlist view are properly validated (http/https only). Image handling uses `BitmapCacheOption.OnLoad` in the right places and freezes `BitmapImage` instances before crossing threads.

Fifteen findings are listed below. Five were addressed during this audit pass (one Critical/High pair on credentials and resource lifecycle, plus a defense-in-depth fix on the updater). The remaining items are Medium/Low and tracked here for follow-up.

Severity	Count	Fixed in this pass	Remaining
Critical	1	1 (mitigated)	0
High	2	2	0
Medium	7	2	5
Low	5	1	4

1. Critical: Hardcoded SQL credentials in `appsettings.json`

File: `appsettings.json` (root)

Description: Both connection strings include plaintext passwords (one for the local SQL Server `WIN-U5IQ2HISNH3`, one for the Azure-hosted catalog DB). If this file is committed to the repository, every developer and the source-control history exposes the credentials. The Azure read-only credential is the lower-risk of the two, but the local-server credential grants write access.

Status: **MITIGATED** - added `.gitignore` entry covering `appsettings.json`, `appsettings.local.json`, `appsettings.Development.json`, plus `bin/`, `obj/`, `Logs/`, `.vs/`,

and `*.log`. Created `appsettings.example.json` as a credential-free template developers can copy.

Recommended next step: Rotate the `JandH` SQL password (and the `tebuddy_readonly` Azure password) since they may have been committed historically. Consider migrating to user-secrets for local dev and Azure Key Vault / environment variables in production.

2. High: `CancellationTokenSource` leak in CardBuildWizardViewModel

File: `ViewModels/CardBuildWizardViewModel.cs` lines 1690-1692, 1848-1850, 1888-1890

Description: Three places in `LoadDecorationItems` / `LoadDecorationStampItems` cancel the existing CTS and immediately reassign a new one. The previous CTS is never disposed, so its internal kernel handle and registered callbacks leak each time the user changes the decoration type. With several wizard sessions open in a row and many type-changes per session, this accumulates.

Status: **FIXED** - added `cts?.Dispose()` immediately after `cts?.Cancel()` in all three sites.

3. High: No `.gitignore` in repository

File: `.gitignore` (did not exist)

Description: Without a gitignore, build output (`bin/` , `obj/`), VS metadata (`.vs/`), local logs, and (most critically) `appsettings.json` were all eligible to be committed.

Status: **FIXED** - new `.gitignore` covers build output, VS state, logs, all `appsettings*.json` variants, publish output, `*.user` , and PDB files.

4. Medium: `ObservableCollection` mutated from async continuations without dispatcher marshalling

File: `ViewModels/CardBuildWizardViewModel.cs` (multiple sites in `LoadDecorationItems` , `LoadDecorationStampItems` , `SearchSentiments` , `SearchInsideSentiments`)

Description: After `await` , code mutates `ObservableCollection` s and assigns to `[ObservableProperty]` backing fields. *In practice* this works because `RelayCommand` dispatches

via the captured `SynchronizationContext`, but if any of these paths are ever called from a non-UI thread (e.g., a future background refresh) it will throw `InvalidOperationException`.

Status: **OPEN**

Suggested fix: Either document the UI-thread requirement explicitly, or wrap the mutation in `Application.Current.Dispatcher.Invoke(...)` defensively in the load methods.

5. Medium: Defense-in-depth path validation for the auto-updater

File: `Services/UpdateService.cs` `ApplyUpdate(...)`

Description: `ApplyUpdate` launches the resolved installer with `UseShellExecute=true`. The path comes from `FindInstaller()` which scans the configured network share, but a symlink/junction inside that share could resolve to an arbitrary binary. The original code did not enforce that the resolved path stays within the trusted directory.

Status: **FIXED** - added a `Path.GetFullPath` containment check: the resolved installer must start with the resolved network base path, otherwise `ApplyUpdate` returns `"Installer path resolved outside the trusted installation folder."` without launching anything.

6. Medium: Sentiment-part decorations were missing from items-used and project build steps

File: `ViewModels/CardBuildWizardViewModel.cs` in `GetSelectedItemIds` and `GetCardBuildSteps`

Description: When configuring an outside sentiment, the user can attach decorations to each part. The decoration's **ink colors** were tracked, but the decoration **item IDs** (and stamp item IDs) were not added to the project's items-used list and did not appear as build steps. Inventory was therefore under-deducted and the project's "items used" view was incomplete.

Status: **FIXED** in the prior audit step (Step 4) - both `AddItem(d.Item.Id)` + stamp-item handling were added in the items-used loop, and matching `"sentiment_decoration"` / `"sentiment_decoration_stamp"` build steps are emitted.

7. Medium: Adhesive items were never recorded in items-used

File: `ViewModels/CardBuildWizardViewModel.cs` `GetSelectedItemIds` + `InitializeAsync`

Description: `AdhesiveItems` was populated as an `ObservableCollection<string>` of names only. The chosen adhesive flowed into each mat/focal/sentiment via the name string, but no inventory item ID was ever added to the project's items-used set, even though adhesives are real inventory items.

Status: **FIXED** in Step 4 - added an `_adhesiveIdByName` dictionary populated during `InitializeAsync`, and an `AddAdhesives(IEnumerable<string>)` helper now feeds the IDs of every adhesive used (BG mats, additional mats, focal parts, sentiments parts, all inside variants) into the items-used result.

8. Medium: Empty `catch` blocks risk hiding bugs

File: `Services/LoggingService.cs` (line ~60), several views (Wishlist URL launch, settings save, image-load fallbacks).

Description: Some `catch { }` blocks intentionally swallow exceptions to keep secondary code paths from killing the app. This is fine in *diagnostic* code (logging itself), but elsewhere it can hide real failures. `LoggingService` already escalates to `Debug.WriteLine`, which is good. Other empty catches should at minimum log to `Debug.WriteLine`.

Status: **OPEN** - audited `LoggingService` already does the right thing (verified during this pass). Other sites bear closer review next pass.

9. Medium: `SelectedDecorationItemType` nullability spread

File: `ViewModels/CardBuildWizardViewModel.cs` + `Views/CardBuildWizardWindow.xaml.cs`

Description: The property was made nullable (`string?`) to fix a WPF ComboBox visual glitch. The ViewModel side updates were comprehensive, but a code-behind `SelectionChanged` handler had unguarded `.Contains` calls and threw `NullReferenceException` when the user picked an item with no type selected (logged in `ERROR_2026-04-26.log` earlier today).

Status: **FIXED** - `DecorationItem_SelectionChanged` now uses `?.Contains(...) ?? false`.

10. Medium: Adhesive picker disappeared after first extra-detail confirm

File: `ViewModels/CardBuildWizardViewModel.cs` `*AdhesiveVisible` properties

Description: Every `*AdhesiveVisible` property included `!*ExtraDetailsDone`, but `*ExtraDetailsDone` becomes `true` as soon as the first decoration is confirmed - permanently hiding the "How was this attached?" picker so the user could never specify the adhesive after adding extras.

Status: **FIXED** - dropped the `!*ExtraDetailsDone` gate on all eight adhesive-visibility properties.

11. Medium: Inside section button visibility lag after confirm/cancel

File: `ViewModels/CardBuildWizardViewModel.cs` `ConfirmCurrentDecoration`, `CancelDecoration`, `RemoveMatDecoration`

Description: Confirm/Cancel decoration only raised `OnPropertyChanged` for the outside variants (`BgMat/AdditionalMat/CardBaseDecorationActive`). The inside variants weren't notified, so the "+ Add Extra Details" / "Add Inside Background Mat" / "Cancel" buttons stayed hidden until something else triggered a re-eval.

Status: **FIXED** - added `InsideBgMatDecorationActive`, `InsideAdditionalMatDecorationActive`, `InsideMatDecorationActive`, `InsideFocalMatDecorationActive`, `SentimentControlsEnabled`, and `MatDecorationActive` notifications to all three commands.

12. Low: Static color-list reload not lock-protected

File: `Services/InventoryService.cs` (lines ~283-300, `ReloadColorOrder` / `ReloadInspirationColors`)

Description: Lazy-loaded static color lists are reloaded by replacing the field reference. Reference assignment is atomic on .NET, but two concurrent reloads could leave one thread reading a partially-loaded list if more than one assignment is involved.

Status: **OPEN**

Suggested fix: wrap the reload-and-assign in a `lock` or use `Lazy<T>`.

13. Low: Tesseract engine recreated per OCR call

> **File:** `Services/SentimentService.cs` (around line 462)
>
> **Description:** A new `TesseractEngine` is constructed and disposed for each OCR pass. That's safe (it's in a `using`) but expensive. Engine startup loads the trained model from disk every time.
>
> **Status:** **OPEN**
>
> **Suggested fix:** cache one engine per language at the service level and dispose on app shutdown via `App.Exit`.

14. Low: `ConfigPathService`-derived paths read without containment validation

> **File:** `Services/InventoryService.cs` (multiple call sites that `File.ReadAllLines(ConfigPathService.X)`)
>
> **Description:** Paths are not user-supplied today, but if `ConfigPathService` ever exposes a "set custom config folder" feature, a path-traversal vector opens up.
>
> **Status:** **OPEN** (low - the threat model assumes the config share is trusted today).

15. Low: Wishlist URL launch (intentional, verified)

> **File:** `Views/WishlistView.xaml.cs` lines 17-29
>
> **Description:** Validates the user-clicked text as an absolute URI, requires the scheme to be exactly `http` or `https`, then launches via `Process.Start(... UseShellExecute=true)`. This is the correct pattern.
>
> **Status:** **VERIFIED OK** - no fix needed.

Files Modified During This Audit

- `.gitignore` - new file, covers credentials/build/logs.

- `appsettings.example.json` - new credential-free template.
- `ViewModels/CardBuildWizardViewModel.cs` - `Dispose()` on the two `CancellationTokenSource` fields before reassignment (3 sites).
- `Services/UpdateService.cs` - path-containment check on the resolved installer.

Build status after fixes: **0 errors, 0 new warnings.**